

Design and Implementation of a Bookstore Order Management System

Ethan Schroder

PostgreSQL · Relational Database Design · PL/pgSQL Triggers · Python
Application Development · Secure Parameterised SQL

Contents

1	Introduction	3
1.1	Project Overview	3
2	Relational Schema Design	4
3	Trigger-Based Business Logic	5
3.1	Why Business Logic Lives in the Database	5
3.2	Statement-Level versus Row-Level Triggers	6
4	Object-Oriented Python Client	7
4.1	Class Design	7
4.2	Design Principles	8
5	Security and Data Integrity	9
5.1	Parameterised Queries	9
5.2	Credential Management	9
6	Testing and Validation	10
6.1	Environment	10
6.2	Schema and Trigger Verification	10
6.3	Client Verification	10
7	Limitations and Future Work	11
8	Conclusion	12

1 Introduction

Relational databases remain the backbone of most transactional business systems: retail ordering, insurance claims processing, and banking ledgers are all, at their core, a small number of normalised tables with carefully enforced integrity rules. What distinguishes a robust implementation from a fragile one is rarely the choice of tables, but rather where the business rules live, how consistently they are enforced, and how safely the application layer is permitted to talk to the database at all.

This report documents a small bookstore order management system built to explore exactly these questions. The system tracks a catalogue of books, a set of customers, and the orders placed between them, while automatically maintaining running sales totals, customer balances, and a full audit trail of order activity. Rather than placing this bookkeeping logic in application code, it is implemented as a set of PostgreSQL triggers and functions, so that the rules hold regardless of which client writes to the database. A Python client, built around a single class with one method per business transaction, sits on top of this schema and is used to drive both interactive use and batch transaction scripts.

1.1 Project Overview

The system is organised into three layers: a relational schema with declarative constraints (Section 2), a set of database triggers and functions that implement the business rules (Section 3), and an object-oriented Python client that talks to the schema exclusively through parameterised queries (Section 4). Section 5 discusses the security and credential-handling decisions taken throughout. Section 6 presents the results of running the full pipeline, schema through client, against a live PostgreSQL instance. Section 7 discusses a genuine limitation of the trigger design uncovered during that testing, and Section 8 concludes.

This project originated as coursework for a relational databases module and has since been reworked as a standalone, portfolio-ready piece: university-specific material and identifiers have been removed, the Python client has been rebuilt around parameterised queries and a proper class-based interface, and the whole system has been re-verified end to end against a fresh database instance rather than assumed to still work.

2 Relational Schema Design

The schema has five tables. `book` and `customer` are the two core entities; `bookOrder` is the associative table recording which customer ordered which book, when, and in what quantity. `bookOrder_log` and `payment_log` are append-only audit tables, populated entirely by triggers rather than by direct application writes. Figure 1 shows the relationships between them.

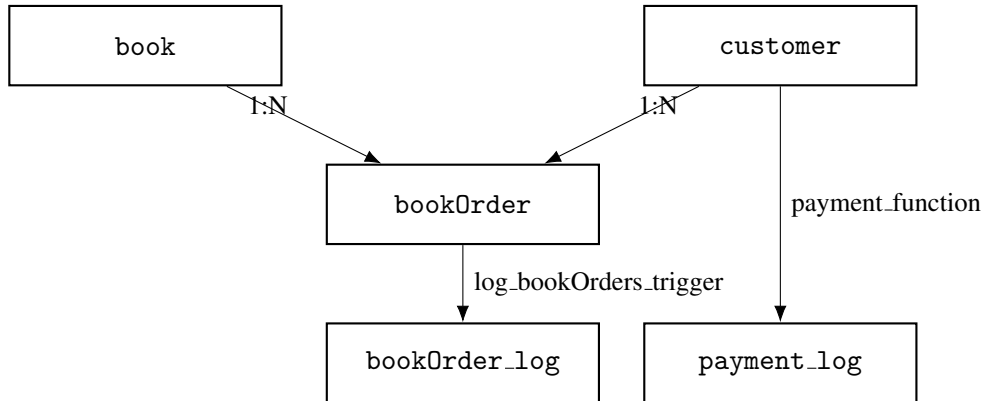


Figure 1: Entity-relationship overview. Solid arrows from `book` and `customer` into `bookOrder` are foreign-key relationships; arrows into the two log tables represent trigger-populated writes rather than foreign keys.

`bookOrder_log` and `payment_log` deliberately have no foreign keys back to the tables they audit. An audit trail should still be able to record that an order for a since-deleted book once existed, so these tables store the raw values rather than referencing rows that may no longer exist.

Table	Constraint	Purpose
<code>book</code>	<code>bno</code> CHECK between 100000–999999	keeps identifiers within a predictable, fixed-width range
<code>book</code>	category CHECK $\in \{\text{Science, Lifestyle, Arts, Leisure}\}$	guarantees the reporting views can group on category safely
<code>customer</code>	<code>cno</code> CHECK between 100000–999999	mirrors <code>book.bno</code> ’s range for a consistent ID scheme
<code>bookOrder</code>	<code>qty</code> CHECK ≥ 1	rejects zero- or negative-quantity orders at the database level
<code>bookOrder</code>	FOREIGN KEY on <code>cno</code> , <code>bno</code> with ON DELETE CASCADE	removing a book or customer cleans up their order history rather than leaving orphan rows

Table 1: Summary of integrity rules enforced declaratively at the schema level, before any trigger logic runs.

3 Trigger-Based Business Logic

3.1 Why Business Logic Lives in the Database

An alternative design would compute `book.sales` and `customer.balance` in the Python client immediately after an order is placed. This was rejected in favour of database triggers for one reason: any client that writes directly to `bookOrder` (the Python client, a future web application, an ad-hoc `psql` session) automatically gets correct sales and balance figures, because the guarantee lives with the data rather than with any particular caller. The trade-off, discussed in Section 7, is that trigger semantics have to be understood precisely, since they behave differently for single-row and multi-row statements.

Trigger / function	Fires on	Effect
<code>log_bookOrders_trigger</code>	any change to <code>bookOrder</code>	writes a row to <code>bookOrder_log</code> tagged INSERT/UPDATE/DELETE
<code>sales_trigger</code>	insert into <code>bookOrder</code>	adds the order quantity onto <code>book.sales</code>
<code>balance_trigger</code>	insert into <code>bookOrder</code>	adds <code>qty * price</code> onto the customer's balance
<code>payment_function</code>	called explicitly	validates the customer exists, then logs a payment to <code>payment_log</code>
<code>payment_trigger</code>	insert into <code>payment_log</code>	subtracts the payment amount from the customer's balance

Table 2: Triggers and functions implementing the schema's business rules.

Four reporting views sit above these tables rather than duplicating logic in the client:

- `current_orders` – orders matching a title fragment
- `book_report` – sales and value by category
- `customer_report` – books ordered per customer
- `customer_bookorders` – order history for a given customer

3.2 Statement-Level versus Row-Level Triggers

`sales_trigger` and `balance_trigger` are declared `AFTER INSERT ON bookOrder` without a `FOR EACH ROW` clause, making them statement-level triggers: PostgreSQL fires them once per statement, not once per affected row [PostgreSQL Global Development Group, 2026]. Both trigger functions determine what changed by reading the single most recent row in `bookOrder_log`.

For a single-row `INSERT` – exactly how `BookstoreClient.place_order` places every order – this is correct: one row is inserted, one log row is written, and the statement-level trigger reads that one row back out. For a multi-row `INSERT INTO bookOrder VALUES (...), (...), (...)`, however, the trigger still fires only once for the whole statement, and only the last row’s contribution to `book.sales` and `customer.balance` is applied. This was caught during testing (Section 6) and is discussed further in Section 7.

4 Object-Oriented Python Client

The Python client is built around a single guiding principle: the caller should never build a SQL string by hand, and the connection lifecycle should never be managed manually. Both are handled once, inside a single class, rather than left to whichever script happens to use the database that day.

4.1 Class Design

BookstoreClient owns exactly one open connection and exposes one method per business transaction: `insert_book`, `delete_book`, `insert_customer`, `delete_customer`, `place_order`, `make_payment`, and the four reporting methods corresponding to the views in Section 3. Table 3 summarises the interface.

Method	Arguments	Returns
<code>insert_book</code>	<code>bno</code> , <code>title</code> , <code>author</code> , <code>category</code> , <code>price</code>	the inserted row
<code>delete_book</code>	<code>bno</code>	the remaining catalogue
<code>insert_customer</code>	<code>cno</code> , <code>name</code> , <code>address</code>	the inserted row
<code>delete_customer</code>	<code>cno</code>	the remaining customer list
<code>place_order</code>	<code>cno</code> , <code>bno</code> , <code>qty</code>	the new order line
<code>make_payment</code>	<code>cno</code> , <code>amount</code>	a status string
<code>search_orders_by_title</code>	<code>title_fragment</code>	matching current orders
<code>customer_order_history</code>	<code>cno</code>	that customer's ordered books
<code>book_sales_report</code>	—	sales and value by category
<code>customer_summary_report</code>	—	books ordered per customer

Table 3: Public methods of BookstoreClient.

The class supports the context manager protocol, so a connection is always closed even if an exception is raised partway through a session:

```
with BookstoreClient() as db:
    db.place_order(cno=900001, bno=234567, qty=2)
    print(db.book_sales_report())
```

4.2 Design Principles

- **Encapsulation.** Connection parameters are resolved once at construction, either from explicit arguments or from environment variables, and are never touched again. No other part of the codebase constructs a connection string.
- **Single responsibility.** Each method implements exactly one business transaction and returns exactly one result. `place_order` inserts an order line and nothing else; it does not update sales or balance figures directly, since that responsibility belongs to the triggers described in Section 3.
- **Consistent interface.** Every write method follows the same shape: set the search path, execute one parameterised statement, and return the resulting rows as a pandas `DataFrame`. A caller does not need to remember a different calling convention for inserts versus deletes versus reports.
- **Command dispatch over branching.** `TransactionRunner`, which drives batches of commands from a script file, maps each single-letter command code to a bound handler method through a dictionary rather than a long `if/elif` chain, making it straightforward to add a new command without touching existing branches.

5 Security and Data Integrity

5.1 Parameterised Queries

Every query issued by `BookstoreClient` is parameterised: values are passed alongside the query text and substituted by the database driver, rather than interpolated into a string beforehand. This is the standard defence against SQL injection recommended by both the Python Database API specification [Lemburg, 1999] and OWASP’s guidance on the subject [OWASP Foundation, 2026]. Concretely, every method in `BookstoreClient` follows the pattern

```
cur.execute(  
    "INSERT INTO book (...) VALUES (%s, %s, ...);",  
    (bno, title, ...)  
)
```

rather than building the statement with Python string formatting. The distinction matters in practice, not only in principle: an earlier version of this client built statements with `.format()`, and a title or address field containing a stray quote character would have been interpreted as SQL syntax rather than as data.

5.2 Credential Management

Database credentials (host, port, database name, user, password) are read from environment variables at runtime rather than being written into source code. A `.env.example` file documents which variables are required without containing real values, and `.env` itself is excluded from version control. This means the repository can be published without exposing any live credential, and rotating a password requires changing one environment variable rather than editing and redeploying code.

6 Testing and Validation

6.1 Environment

The full pipeline, schema creation, trigger and view definitions, seed data, and the Python client, was executed end to end against a local PostgreSQL 16 instance rather than assumed correct from inspection alone. This surfaced two genuine defects, both described below, before either could reach a reader of this report or a user of the repository.

6.2 Schema and Trigger Verification

After loading the seed data in `04_sample_data.sql`, the reporting views were queried directly. Table ?? reproduces the actual output of `book_report` at that point.

Category	Number of sales	Total value
Leisure	34	373.66
Arts	11	205.91
Lifestyle	1	14.99
Science	12	420.98

Table 4: `book_report` output after loading the seed data (actual test output).

The total of 58 copies sold across all categories matched the sum of quantities across the 19 seeded orders exactly, confirming that `sales_trigger` correctly attributes each order to the right book when orders are inserted one statement at a time.

6.3 Client Verification

`run_transactions.py` was then run against `sample_transactions.txt`, which exercises every command described in Table 2’s corresponding client methods: inserting and deleting a book, inserting and deleting a customer, placing an order, logging a payment, searching by title fragment, retrieving a customer’s order history, and both summary reports. All ten commands completed without error and produced results consistent with the direct SQL verification above; the payment command reduced the paying customer’s balance by exactly the logged amount, and the order-placement command increased `book.sales` for the ordered title by the ordered quantity.

7 Limitations and Future Work

The most significant limitation is the one identified in Section 3.2: because `sales_trigger` and `balance_trigger` are statement-level rather than row-level, a multi-row `INSERT` into `bookOrder` silently updates aggregates for only the last row in the batch. This does not affect the application as currently used, since `BookstoreClient.place_order` always issues single-row inserts, and the seed data was rewritten during testing to do the same. It would, however, affect any future bulk-import path (for example, loading historical orders from a CSV export) unless the trigger functions are rewritten to reference `NEW` directly under a `FOR EACH ROW` trigger rather than querying `MAX(bookOrder_logid)`.

A second limitation is the housekeeping statement that deletes `bookOrder` rows older than a month: it is run manually rather than scheduled, since the environment has no job scheduler configured. In a production deployment this would be automated via `pg_cron` or an equivalent extension.

Finally, the reporting views recompute their results from base tables on every query rather than being materialised. For the data volumes here this is immaterial, but a much larger `bookOrder` table would likely justify materialised views refreshed on a schedule, particularly for `book_report` and `customer_report`, which aggregate across the full order history.

8 Conclusion

This project implements a small but complete order management system: a normalised relational schema with declarative integrity constraints [Codd, 1970, Silberschatz et al., 2019], business rules enforced through database triggers rather than application code, and an object-oriented Python client that never builds a SQL statement by hand. Testing the full pipeline against a live PostgreSQL instance, rather than relying on inspection alone, surfaced two concrete defects: column widths too narrow for realistic data, and a statement-level trigger design that silently mishandles batched inserts. Both are now documented and, in the first case, fixed; the second is scoped precisely enough to be addressed cleanly if a bulk-import feature is ever added.

The broader lesson generalises beyond this specific schema. Encapsulating business rules in the database, rather than in whichever client happens to be writing to it that day, only pays off if the trigger semantics are understood exactly, statement-level versus row-level is not a detail that can be safely glossed over. Encapsulating query construction in a single class with a consistent, parameterised interface has a more straightforward payoff: no caller of `BookstoreClient` can accidentally reintroduce a SQL injection vector, because no caller is ever given the opportunity to build a query string in the first place [Meyer, 1988].

References

- [Codd, 1970] Codd, E. F. (1970). A relational model of data for large shared data banks. *Communications of the ACM*, 13(6):377–387.
- [Lemburg, 1999] Lemburg, M.-A. (1999). PEP 249 – Python Database API Specification v2.0. Python Enhancement Proposals.
- [Meyer, 1988] Meyer, B. (1988). *Object-Oriented Software Construction*. Prentice Hall.
- [OWASP Foundation, 2026] OWASP Foundation (2026). SQL Injection Prevention Cheat Sheet. OWASP Cheat Sheet Series. Accessed July 2026.
- [PostgreSQL Global Development Group, 2026] PostgreSQL Global Development Group (2026). CREATE TRIGGER. PostgreSQL Documentation. Accessed July 2026.
- [Silberschatz et al., 2019] Silberschatz, A., Korth, H. F., and Sudarshan, S. (2019). *Database System Concepts*. McGraw-Hill Education, 7 edition.